DBMS Lecture 5: Three-Schema Architecture

Three Levels of Data Abstraction & Data Independence

Topper's Revision Notes | Visual Diagrams, GATE Shortcuts & Interview Q&A

---

1. Introduction

The **Three-Schema Architecture** is one of the most fundamental concepts in Database Management Systems. It is also referred to as:

- Three-Level Architecture
- Three-Schema Architecture
- Three Levels of Data Abstraction

The architecture divides the database system into three distinct abstraction levels:

1. **External Level** (External Schema / View Level)
2. **Conceptual Level** (Conceptual Schema / Logical Level)
3. **Internal Level** (Internal Schema / Physical Level)

MAIN PURPOSE (★)

The primary goal of the Three-Schema Architecture is to separate user applications from the physical database, thereby achieving **Data Abstraction** and supporting **Data Independence**.

2. Quick Refresher: What is a Schema?

Recall that a **Schema** is the *logical structure or design* of the database.

For example, given student data records, the schema defines the blueprint:

```
Student Table Schema
   Roll_No  -->  Integer (Primary Key)
   Name     -->  Varchar (String)
   Age      -->  Integer
```

A Schema defines:

- Attributes / Columns & Data Types
- Tables & Entity Relationships
- Integrity Constraints & Domain Rules

3. Why Do We Need Three Levels? (The Gmail Analogy)

Users should **never** need to understand low-level physical disk storage to interact with a database.

Real-World Example (Gmail Inbox): When you check your Gmail inbox, you see:

```
Inbox  -->  [Mail 1 | Mail 2 | Mail 3 | Mail 4]
```

You do **NOT** need to know:

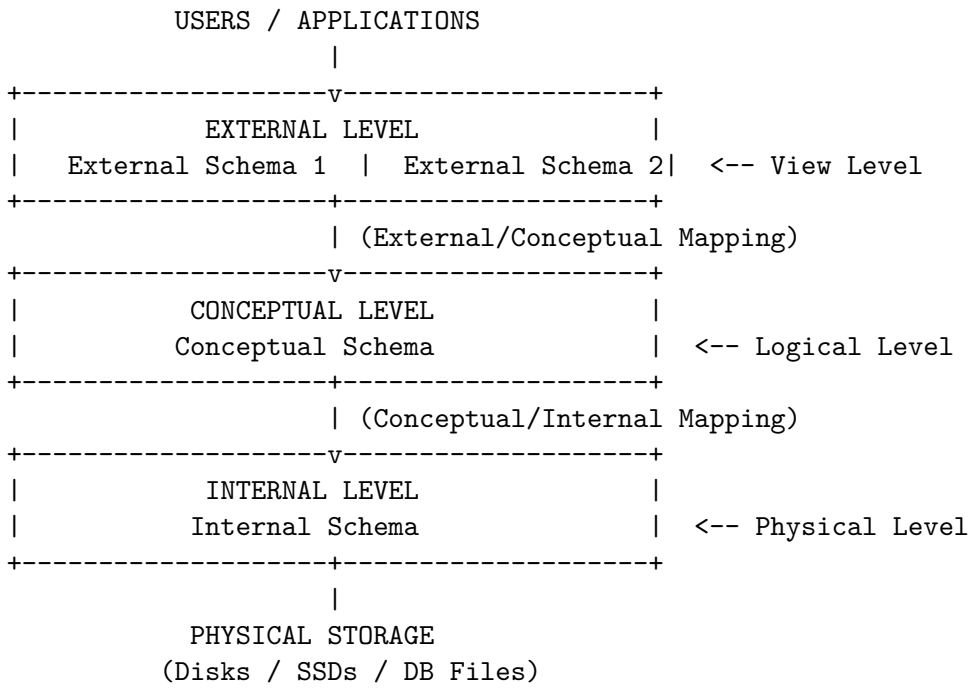
- Which physical data center or server stores your emails.
- Which physical hard drive, SSD, or disk block holds the message data.
- How B-Trees or indexing files retrieve records internally.

You simply request: *"Show me my emails."* The complex implementation details are hidden behind levels of abstraction.

Data Abstraction Definition

Data Abstraction is the process of hiding complex internal storage/implementation details from end-users while exposing only the essential interface needed for their specific operations.

4. The Three-Schema Architecture Structure



Quick Memory Trick

External Level $\longrightarrow$ Conceptual Level $\longrightarrow$ Internal Level

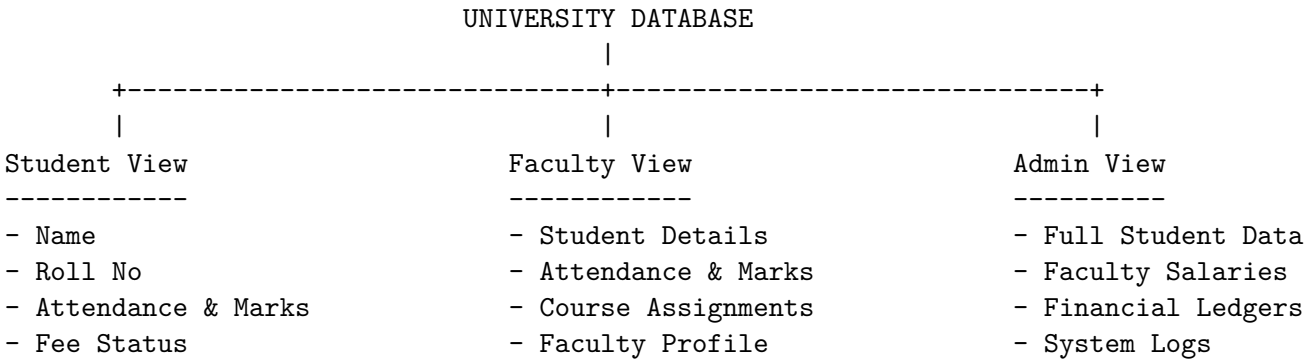
(View Level) $\longrightarrow$ (Logical Level) $\longrightarrow$ (Physical Level)

5. Level 1 — External Schema (View Level)

- **Also Called:** External Level, View Level, User Level.
- **Definition:** Describes how a specific user or group of users perceives the database.
- **Purpose:** Provides customized, tailored views of the database for different user roles while hiding irrelevant or sensitive data.

Why External Schemas are Essential for Security & Privacy

Consider a University Database:



Security & Authorization

By defining separate External Schemas, a student is strictly restricted from viewing faculty salaries or administrative financial data. This supports security, authorization, data privacy, and clean user interfaces.

6. Level 2 — Conceptual Schema (Logical Level)

- **Also Called:** Conceptual Level, Logical Level, Global Logical Schema.
- **Definition:** Describes the **overall logical structure of the entire database** for all users.
- **Key Focus:** Entities, attributes, relationships, data types, business rules, and security constraints.
- **Abstraction:** Hides physical storage layouts while maintaining a single unified logical blueprint.

Conceptual Schema as a Building Blueprint

- **Architectural Blueprint:** Defines room locations, doors, walls, and plumbing lines before building.
- **Conceptual Schema:** Defines entities (`Student`, `Course`, `Faculty`, `Department`), attributes, primary keys, and relationships (`Student Enrolls In Course`) before physical creation.

ER Modeling Connection (★)

Entity-Relationship (ER) Modeling is the primary conceptual design tool used to construct the conceptual schema before mapping it into relational tables.

7. Level 3 — Internal Schema (Physical Level)

- **Also Called:** Internal Level, Physical Level, Physical Schema.
- **Definition:** Describes **how data is physically stored and organized** inside storage media.
- **Lowest Level:** Deals with database files, page allocations, block sizes, record clustering, indexes (B-Trees, Hash Indexes), and compression algorithms.

Logical Tables vs. Physical Disk Layout

- **Logical Table (User View):** A neat table with rows and columns.
- **Physical Storage (DBMS Level):** Data split across disk pages, binary blocks, offset pointers, and index nodes.

Role of the Database Administrator (DBA)

The **DBA** works extensively at the Internal Level to tune storage allocation, create indexes, optimize disk I/O performance, configure backup/recovery plans, and manage hardware partitioning.

8. Core Concept — Three Levels of Data Abstraction

Level Name	Alternate Name	Main Concern & Perspective
External Level	View Level	What a **specific user / application** sees.
Conceptual Level	Logical Level	What the database **logically contains** as a whole.
Internal Level	Physical Level	How the data is **physically stored** on hardware.

Table 1: Summary of Data Abstraction Levels

Memory Summary

- **External:** *"What do I (as a user) see?"*
- **Conceptual:** *"What entities and relationships exist in total?"*
- **Internal:** *"How are bytes and disk pages organized?"*

9. Data Independence (★★★ CRITICAL FOR EXAMS)

Definition of Data Independence

Data Independence is the capacity to modify the schema definition at one level of a database system without altering the schema definition at the next higher level.

There are **two types** of Data Independence:

1. Physical Data Independence
2. Logical Data Independence

9.1 Physical Data Independence

- **Definition:** The ability to modify the **Internal Schema** without requiring changes to the **Conceptual Schema**.
- **Why it matters:** You can switch hard drives, change file access methods, add new B-Tree indexes, or re-organize disk pages without breaking logical table definitions or application queries.

9.2 Logical Data Independence

- **Definition:** The ability to modify the **Conceptual Schema** without requiring changes to **External Schemas** or user application views.
- **Why it matters:** You can add new attributes (`Email` column to `Student`), add new tables (`Scholarships`), or alter table relationships without forcing modifications to existing user interfaces or applications that do not use those new fields.

Comparison: Physical vs. Logical Data Independence

10. Database Mappings Between Levels

The three levels are connected through **DBMS Mappings** that dynamically translate requests:

```
[ External Schema (View) ]
           |
```

Dimension	Physical Data Independence	Logical Data Independence
Level Changed	Internal / Physical Level	Conceptual / Logical Level
Protected Level	Conceptual Level	External Levels / User Views
Typical Operations	Adding indexes, changing file structures, switching storage media.	Adding columns/tables, splitting entities, modifying constraints.
Difficulty	**Easier** to achieve.	**Harder** to achieve (due to view dependencies).
Primary Goal	Hide physical disk storage changes.	Hide logical schema evolution from users.

Table 2: Physical vs. Logical Data Independence

```
(External / Conceptual Mapping)
            |
[ Conceptual Schema (Logical) ]
            |
(Conceptual / Internal Mapping)
            |
[ Internal Schema (Physical) ]
```

- **External/Conceptual Mapping:** Maps objects in the user view to conceptual tables.
- **Conceptual/Internal Mapping:** Maps conceptual tables to physical files, block offsets, and storage paths.
- **Overhead Note:** Mappings introduce slight processing overhead during query execution, but provide immense security and flexibility.

11. Role Assignment Across Architecture Levels

Level	Primary Stakeholders	Main Responsibilities
External Level	End Users, UI Designers, Software Engineers	Crafting clean application UI, restricting access, building user features.
Conceptual Level	Database Designers, System Analysts	Structuring entities, normalization, ER design, defining integrity constraints.
Internal Level	Database Administrators (DBAs), Storage Engineers	Index optimization, disk allocation, RAID setup, backup & recovery strategies.

12. Common Misconceptions: Relational Table vs. Physical File

CRITICAL DISTINCTION

A Relational Table is a Logical Structure, NOT a Physical File!

- Users see: `Student(Roll_No, Name, Age)`.
- DBMS disk storage: May partition rows across multiple physical disk files, store attributes in columnar files, or spread data across clustered nodes.

Never assume a 1:1 match between a logical table and a single file on disk!

13. Top Interview & GATE Questions

Q1. What is the Three-Schema Architecture?

Answer: It is an architectural framework that separates database systems into three levels (External, Conceptual, Internal) to provide data abstraction and support physical/logical data independence.

Q2. What is Data Abstraction?

Answer: Data abstraction is the practice of suppressing low-level internal storage and implementation details while presenting clear, simplified interfaces tailored to user needs.

Q3. Differentiate between Physical and Logical Data Independence.

Answer: Physical Data Independence protects the conceptual schema from changes made to physical storage structures. Logical Data Independence protects external user views from changes made to the global conceptual schema.

Q4. Which type of data independence is harder to achieve?

Answer: Logical Data Independence is harder to achieve because user applications are often tightly coupled to logical attributes and table definitions.

Q5. Does every user see the same external schema?

Answer: No. Multiple external schemas can exist simultaneously, giving different user roles (e.g., Student vs. Faculty vs. Admin) distinct access levels and views.

14. GATE & Rapid Revision Flashcards

Quick Memory Formula

Three-Schema Architecture $\Longrightarrow$ Data Abstraction $\Longrightarrow$ Data Independence

- **View Level = External Schema** $\rightarrow$ What the user sees.
- **Logical Level = Conceptual Schema** $\rightarrow$ What data exists in total.
- **Physical Level = Internal Schema** $\rightarrow$ How data is stored on disk.
- **Internal Changes** $\rightarrow$ Handled by **Physical Data Independence** (Conceptual unaffected).
- **Conceptual Changes** $\rightarrow$ Handled by **Logical Data Independence** (External views unaffected).

DBMS Study Roadmap Progress

- Lecture 1: Database $\rightarrow$ DBMS $\rightarrow$ RDBMS
- Lecture 2: File System vs DBMS
- Lecture 3: 2-Tier & 3-Tier Architecture
- Lecture 4: Schema in Database
- **Lecture 5: Three-Schema Architecture & Data Abstraction (COMPLETED)**
- *Next Lecture: Data Independence Deep-Dive, Data Models & Entity-Relationship (ER) Diagrams*